NestJS JWT Auth — Learning Plan

A step-by-step path to fully understand the project, written for developers coming from Laravel.

> Learn in the order a request flows through the app, and map each piece to its Laravel equivalent as you go. Total time: about 5 hours, including exercises.

0. Laravel → NestJS map (keep open while reading)

Laravel	NestJS	In this project
`routes/api.php` + Controller	Controller with route decorators (`@Get` , `@Post`)	`auth.controller.ts`
Service container / `bind()`	Module `providers` + constructor injection	`auth.module.ts`
ServiceProvider	Module (`@Module`)	`*.module.ts`
FormRequest `rules()`	DTO class + class-validator decorators + `ValidationPipe`	`dto/*.ts` , `app.setup.ts`
Middleware `auth:sanctum`	Guard	`jwt-auth.guard.ts`
Auth guard driver (Sanctum/Passport)	Passport **Strategy**	`jwt.strategy.ts`
`Gate` / `can:` middleware	RolesGuard + `@Roles()`	`roles.guard.ts`
`withoutMiddleware('auth')`	`@Public()` metadata	`public.decorator.ts`
`$request->user()` / `auth()->user()`	`req.user` / `@CurrentUser()`	`current-user.decorator.ts`
`Hash::make` / `Hash::check`	`bcrypt.hash` / `bcrypt.compare`	`auth.service.ts`
`$hidden = ['password']`	`toPublic()` strips the hash manually	`users.service.ts`
Exception Handler `render()`	Exception Filter	`http-exception.filter.ts`
`.env` + `config()`	`ConfigModule` + `ConfigService`	`app.module.ts`
`bootstrap/app.php` / `public/index.php`	`main.ts`	`main.ts`
Feature tests (`$this->postJson`)	e2e tests with supertest	`test/auth.e2e-spec.ts`

Big mental shift: Laravel uses *arrays and conventions* (such as route files and middleware groups). NestJS uses *decorators and metadata*: `@Something()` on a class or method attaches data that Nest reads at runtime.

1. Boot sequence (~20 min)

1. `src/main.ts`: creates the app, registers the pipe and filter, sets up Swagger, listens on the port.
2. `src/app.setup.ts`: global `ValidationPipe` (similar to every FormRequest running automatically) and the error filter.
3. `src/app.module.ts`: the root module. Note the `APP_GUARD` entries, which put every route behind auth by default.

> **Exercise:** Comment out the `JwtAuthGuard` line and call `/auth/profile`. What changes? Restore it afterwards.

2. Modules + Dependency Injection (~30 min)

1. `users/users.module.ts`: `providers` registers the service and `exports` makes it available to other modules. Laravel has nothing like this: every binding there is global.
2. `auth/auth.module.ts`: imports `UsersModule`, which is why `AuthService` can inject `UsersService`.
3. `auth/auth.service.ts` constructor: `private readonly users: UsersService` is constructor injection, like Laravel's auto-resolved type hints.

> **Exercise:** Remove `exports: [UsersService]` and run the app. Read the DI error. You will see it often in Nest.

3. Request lifecycle — the key part (~45 min)

Nest's order: **Middleware → Guards → Interceptors → Pipes → Controller → Interceptors → Exception Filters (on error)**.

Trace `GET /auth/profile` with a valid token:

1. `jwt-auth.guard.ts`: `canActivate` checks the `@Public()` metadata via `Reflector`. It's absent, so it calls `super.canActivate`, which hands over to Passport.
2. `jwt.strategy.ts`: the constructor config tells Passport where to find the token (the Bearer header) and which secret to verify it with. `validate()` runs only if the token is valid, and its return value becomes `req.user`.
3. `roles.guard.ts`: no `@Roles` on this route, so it passes.
4. `auth.controller.ts` `profile()`: `@CurrentUser()` reads `req.user`.

Then trace the failure paths:

- No token → Passport throws 401 → `http-exception.filter.ts` formats the response.
- `POST /auth/signup` with a bad email → `ValidationPipe` throws 400 before the controller runs.

> **Exercise:** Add `console.log` in the guard, the strategy's `validate()`, and the controller. Call the endpoint and check the order of the logs.

4. Decorators (~20 min)

Read the three files in `auth/decorators/`:

- `SetMetadata`: attaches data to a route (`@Public`, `@Roles`).
- `Reflector.getAllAndOverride`: guards use it to read that data. It checks the method first, then the class.
- `createParamDecorator`: a custom parameter injector (`@CurrentUser`).

> **Exercise:** Put `@Public()` on the `AuthController` *class* instead of on each method. What happens to `/auth/profile`, and why?

5. Business logic (~30 min)

`auth.service.ts`:

- `signup`: checks for a duplicate email, hashes the password with bcrypt, strips the password from the response.
- `login`: returns the same error message for an unknown email and a wrong password, which prevents user enumeration.
- `issueTokens`: two tokens signed with **different secrets**.
- `refresh`: re-reads the user, so a deleted user can't refresh.

Then read `jwt-refresh.strategy.ts`: the same Strategy pattern with a different name (`'jwt-refresh'`), token source, and secret.

6. Validation + errors (~20 min)

- `dto/signup.dto.ts`: decorators serve as validation rules. The `@Api*` decorators are only for Swagger.
- `app.setup.ts`: `whitelist` + `forbidNonWhitelisted` reject unknown fields. That's why sending `role` to `/auth/login` returns 400.
- `http-exception.filter.ts`: `@Catch()` with no argument catches everything and returns `{ success: false, statusCode, message, path, timestamp }`.

7. Tests (~20 min)

`test/auth.e2e-spec.ts` is close to a Laravel feature test. `Test.createTestingModule` boots the app and supertest sends the HTTP calls. Run it with `npm run test:e2e`.

> **Exercise:** Add a test showing that `/auth/admin` without any token returns 401 (not 403).

8. Hands-on (~1 hr)

Run `npm run start:dev`, then:

1. Use Swagger at `http://localhost:3000/docs`: signup → login → **Authorize** → profile.

2. Decode the access token at jwt.io and find `sub`, `email`, `role`, `exp`.
3. Set `JWT_ACCESS_EXPIRES=20s`, restart, and wait for the token to expire. Then call `/auth/refresh`.

9. Build something yourself

Add `PATCH /auth/password` (protected) with a `ChangePasswordDto` (`oldPassword`, `newPassword`). Verify the old password, hash the new one, and write a test for it. If you can do that without looking back, you understand the project.

Gotchas coming from Laravel

- Imports end in `.js` even though the files are `.ts`, because the project uses ES modules. Always write `.js`.
- `import type` is required for interfaces used in decorated method parameters (a TypeScript rule).
- There's no Eloquent here; users live in an in-memory array. The real-world equivalent would be TypeORM or Prisma.
- Services are singletons by default. That's why the in-memory array persists between requests. In Laravel, each request starts fresh.
- Guards return `true` / `false` or throw. There's no `$next($request)` like in Laravel middleware.

> **Before an interview, reread in this order:** strategy → guard → public decorator → roles guard → filter. Those five files answer the self-check questions.